



Text Representation and Pre-Processing Text

IFN647- Week 2

Professor Yuefeng Li
School of Computer Science
Queensland University of Technology

Learning Objectives

Topic ID	Key topics	Mapping to slides	Activities
1	Text Representation	2- 5	
2	Preprocessing text: <i>Text Statistics & Document Parsing</i>	6-14	Q1
3	Noise Reduction: <i>Stopping and Stemming</i>	15-20	Q2
4	Feature Extraction: <i>Phrases & N-grams</i>	21-25	Q3
5	Document Structure and Markup: HTML tags & Hyperlinks	26-27	Q4
6	Review Questions and Workshops	28-29	
	References	30	

1. Text Representation

- Text representation converts raw, unstructured human language into machine-understandable formats (e.g., numerical vectors or SQL) for NLP, enabling algorithms to perform tasks like classification, sentiment analysis, and search.
- Why Text Representation Matters
 - Computers do not understand the meaning of raw text; they can only process text after it is encoded into machine-readable representations (e.g., numbers or structures).
 - Machine learning models require numerical inputs.
 - Text representation bridges human language and machine learning.
 - Quality of representation → quality of model performance.

Challenges of Text Data

- Unstructured and high-dimensional
 - Text data does not follow a fixed schema like tables or spreadsheets.
 - Each unique word can be treated as a feature, leading to thousands or millions of dimensions.
- Vocabulary size can be very large
- Synonyms and polysemy (multiple meanings - ambiguity)
 - Synonyms: Different words can have similar meanings (e.g., car and automobile), but traditional methods treat them as unrelated features.
 - Polysemy: The same word can have different meanings depending on context (e.g., national bank of Australia vs west riverbank).
- Word order and context matter
 - Many traditional representations ignore order and context, motivating more advanced models such as n-grams and transformers.

Example: Text-to-SQL

- Text-to-SQL is a long-standing task in NLP, aiming to convert natural language (NL) questions into database-executable SQL queries [1].
- Early traditional efforts were mostly based on well-designed rules and templates
 - Challenges in Linguistic Complexity and Ambiguity:
 - Schema Dependency and Poor Portability (Table names, Column names and Schema structure)
 - Vocabulary Mismatch: Synonyms not captured, Abbreviations and aliases ignored and Domain terminology varies.
 - Ambiguity in Natural Language: e.g., “top students” → by GPA? or by ranking?

CBICOS No.00213.1

” Ask ChatGPT

Natural Language Question

“Could you tell me the names of the 5 leagues with the highest matches of all time and how many matches were played in the said league?”

Assumed Database Schema (typical)

- **League**(league_id , league_name)
- **Match**(match_id , league_id , ...)

(Each match belongs to one league via league_id .)

SQL Query

```
sql

SELECT
    l.league_name,
    COUNT(m.match_id) AS total_matches
FROM League l
JOIN Match m
    ON l.league_id = m.league_id
GROUP BY l.league_name
ORDER BY total_matches DESC
LIMIT 5;
```

Example: Text-to-SQL cont.

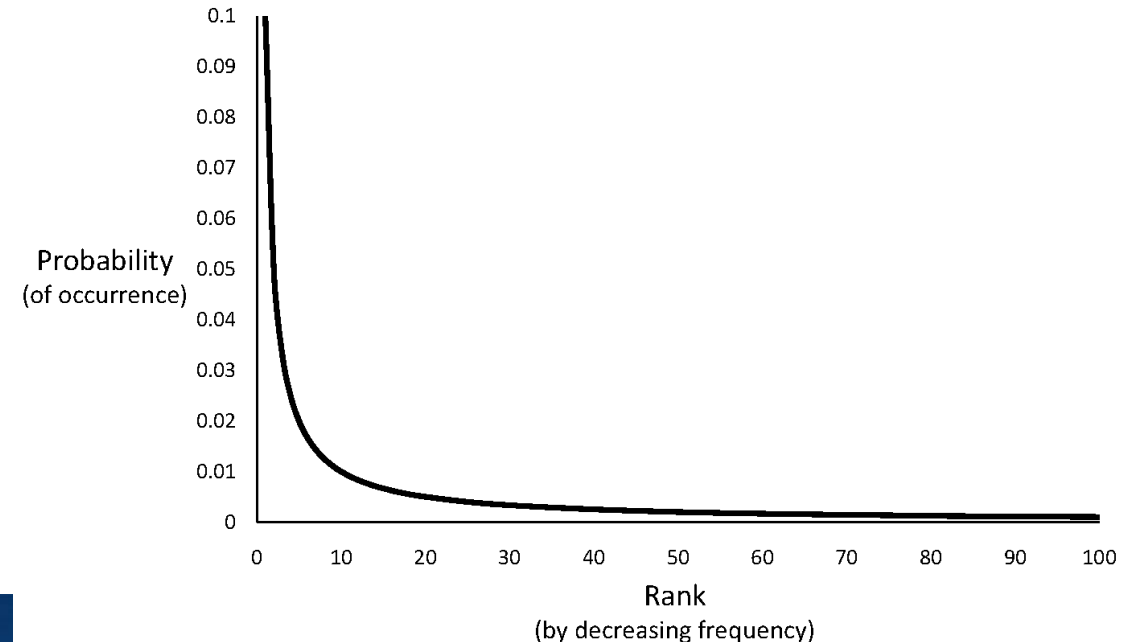
- Next-Generation Database Interfaces: LLM-Based Text-to-SQL
 - Challenges [1]:
 - Users often pose unclear or informal questions (containing ambiguity) that do not exactly match database values; real-world queries may include synonyms, typos, and vague expressions.
 - In addition, real-world datasets are typically much smaller than research benchmark datasets.
 - Future Extension [1]:
 - Databases encode relational knowledge through their structural information, and structure-based question answering can potentially enhance text-to-SQL systems.

2. Precocessing Text: *Text Statistics & Document Parsing*

- We start with a web search problem for a give query Q to understand how to process text.
- Precocessing text is to convert documents to *index terms*.
- Why?
 - Matching the exact string of characters in Q (typed by a user) is too restrictive
 - i.e., it doesn't work very well in terms of effectiveness
 - Not all words are of equal value in a search
 - Sometimes not clear where words begin and end
 - Not even clear what a word is in some languages
 - e.g., Chinese, Korean

Processing Text: Text Statistics

- Before discussing document parsing, we first examine the statistical characteristics of word occurrences in documents. This is a standard practice in information retrieval and NLP, as text collections typically contain large vocabularies,
- Many properties of word occurrence and frequency distributions exhibit statistically predictable patterns.
 - e.g., distribution of word counts
- Search or retrieval models and ranking algorithms depend heavily on statistical properties of words
 - e.g., important words occur often in documents but are not high frequency in collection
- **Zipf's Law** - Distribution of word frequencies is very *skewed*
 - a few words occur very often, many words hardly ever occur
 - e.g., two most common words (“the”, “of”) make up about 10% of all word occurrences in text documents



Vocabulary Growth

- Another important problem in representing text documents computationally is predicting the vocabulary likely to appear in a given dataset.
- As corpus grows, so does vocabulary size
 - Fewer new words when corpus is already large
- Observed relationship (*Heaps' Law*):

$$v = k \cdot n^{\beta}$$

where v is vocabulary size (number of unique words),

n is the number of words in corpus,

k, β are parameters that vary for each corpus (typical values given are $10 \leq k \leq 100$ and $\beta \approx 0.5$)

Processing Text: Document Parsing

- Document parsing involves the recognition of the content and structure of text documents.
- Forming words from sequence of characters is called **tokenizing**.
- Surprisingly complex in English, can be harder in other languages
- **Definition of Words in Early IR systems:**
 - any sequence of alphanumeric characters of length 3 or more
 - terminated by a space or other special character
 - upper-case changed to lower-case

Tokenizing Example

- Example:
 - “Bigcorp's 2007 bi-annual report showed profits rose 10%.” $\Rightarrow$
 “bigcorp 2007 annual report showed profits rose”
- Too simple for search applications or even large-scale experiments.
- Why? Too much information lost, e.g.,
 - “bi-annual” (means in two years) is different with “annual” (in one year).
 - Missed important value “10%”.
- **Small decisions in tokenizing can have major impact on effectiveness of some queries [2]**

Tokenizing Problems

- Small words can be important in some queries, usually in combinations
 - xp, ma, pm, ben e king, el paso, master p, gm, j lo, world war II, VW (Volkswagen)
- Both hyphenated and non-hyphenated forms of many words are common
 - Sometimes hyphen is not needed
 - e-bay, wal-mart, active-x, cd-rom, t-shirts
 - At other times, hyphens should be considered either as part of the word or a word separator
 - winston-salem, mazda rx-7, e-cards, pre-diabetes, t-mobile, spanish-speaking

Tokenizing Problems cont.

- Special characters are an important part of tags, URLs, code in documents
- Capitalized words can have different meaning from lower case words
 - Bush, Apple
- Apostrophes can be a part of a word, a part of a possessive, or just a mistake
 - rosie o'donnell, can't, don't, 80's, 1890's, men's straw hats, master's degree, england's ten largest cities, shriner's.
- Numbers can be important, including decimals
 - nokia 3250, top 10 courses, united 93, quicktime 6.5 pro, 92.3 the beat, 288358
- Periods can occur in numbers, abbreviations, URLs, ends of sentences, and other situations
 - I.B.M., Ph.D., cs.umass.edu, F.E.A.R.

Note: tokenizing steps for queries **must be identical** to steps for documents [2].

Tokenizing Process

- The first step is to use a parser (for a specific markup language, e.g., HTML) to identify relevant parts of the document, first words, then terms, for tokenization.
- Examples of definitions:
 - **Word:** Any sequence of alphanumeric characters, terminated by a space or a special character.
 - **Term:** A word converted entirely to lower case.
- Defer complex decisions to other components.
 - Do you want to index everything?
 - Example: 92.3 → 92 3 but search finds documents with 92 and 3 adjacent
 - incorporate some rules to reduce dependence on query transformation components
 - Examples of rules used with TREC conference
 - Apostrophes in words ignored
 - o'connor → oconnor bob's → bobs
 - Periods in abbreviations ignored
 - I.B.M. → ibm Ph.D. → phd

Example 2: Tokenization

```
<?xml version="1.0" encoding="ISO-8859-1"?>
- <newsitem xml:lang="en" date="1997-08-06" id="root" itemid="783802">
  <title>UK: Britain's Channel 5 to broadcast Fashion Awards.</title>
  <headline>Britain's Channel 5 to broadcast Fashion Awards.</headline>
  <dateline>LONDON 1997-08-06</dateline>
  - <text>
    <p>Channel 5, Britain's newest terrestrial television channel, on Wednesday said it struck a
      three-year deal with the British Fashion Council for exclusive rights to broadcast the
      Lloyds Bank British Fashion Awards.</p>
    <p>"Fashion is now a defining part of British popular culture and this was an unmissable
      opportunity for 5 to give the British Fashion Awards its rightful place and profile on
      television," Adam Perry, Channel 5's controller of events, said in a statement.</p>
    <p>Initial, the independent production company that will produce coverage of the awards,
      said it aims to bring in top celebrities and world-class performers for the event.</p>
    <p>The British Fashion Awards will be held on October 22 at London's Royal Albert Hall.</p>
    <p>-- London Advertising Newsdesk +44 171 542 2815</p>
  </text>
  <copyright>(c) Reuters Limited 1997</copyright>
</newsitem>
```

```
>>> import string
>>> line = "<p>The British Fashion Awards will be held on October 22 at London's Royal Albert Hall.</p>"
>>> line = line.replace("<p>", "").replace("</p>", "")
>>> line = line.translate(str.maketrans("", string.digits)).translate(str.maketrans(string.punctuation, ' '*len(string.punctuation)))
>>> line
'The British Fashion Awards will be held on October  at London s Royal Albert Hall '
>>> words=line.split()
>>> words
['The', 'British', 'Fashion', 'Awards', 'will', 'be', 'held', 'on', 'October', 'at', 'London', 's', 'Royal', 'Albert', 'Hall']
>>> terms=[word.lower() for word in words if len(word)>2]
>>> terms
['the', 'british', 'fashion', 'october', 'royal', 'hall', 'will', 'albert', 'london', 'held', 'awards']
```

- Words are fundamental constructs in many natural languages.
- A term is a label for one specific concept, which can contain a single word (simple term) or many words (complex term). A term can also be a symbol (©, ®) or a formula (H₂O).

3. Noise Reduction

- It is the process of cleaning raw text to remove irrelevant or distracting information (noise) and transforming the text into meaningful representations (features) that can be used for analysis or modeling.
- This ensures that only informative content contributes to downstream tasks like classification, clustering, or information retrieval (IR).
- Basic Steps
 - Stopword Removal
 - Purpose: Remove common words that carry little semantic value.
 - Example: ["machine", "learning", "is"] → ["machine", "learning"]
 - Stemming / Lemmatization
 - Purpose: Reduce words to their base or root form to consolidate variations.
 - Example: ["running", "runs", "ran"] → ["run", "run", "run"]
 - Special phrase or n-gram extraction.
 - Purpose: Capture context beyond individual words.
 - Example: Text: "I love natural language processing" →
Unigrams: ["I", "love", "natural", "language", "processing"]
Bigrams: ["I love", "love natural", "natural language", "language processing"]
Trigrams: ["I love natural", "love natural language", "natural language processing"]

Stopword Removal

- Stopwords are function words (determiners, prepositions) have little meaning on their own; and
- High occurrence frequencies
- Treated as *stopwords* (i.e., removed)
 - reduce index space, improve response time, improve effectiveness
- Can be important in combinations
 - e.g., “to be or not to be”
- Stopword list can be created from high-frequency words or based on a standard list.
- Lists are customized for applications, domains, and even parts of documents
 - e.g., “click” is a good stopwords for anchor text
- **Best policy** is to index all words in documents, make decisions about which words to use at query time.

Stemming / Lemmatization

- Many morphological variations of words
 - *inflectional* (plurals, tenses)
 - *derivational* (making verbs nouns etc.)
- In most cases, these have the same or very similar meanings
- Stemmers attempt to reduce morphological variations of words to a common stem
 - usually involves removing suffixes
- Can be done at indexing time or as part of query processing (like stopwords).
- Generally, a small but significant effectiveness improvement
 - can be crucial for some languages
 - e.g., 5-10% improvement for English, up to 50% in Arabic.

Stemming Algorithms

- Two basic types
 - Dictionary-based: uses lists of related words
 - Algorithmic: uses program to determine related words
- Algorithmic stemmers
 - *suffix-s*: remove 's' endings assuming plural
 - e.g., cats → cat, lakes → lake, wiis → wii
 - Many *false negatives* (have different stems): triangle → triangular
 - Some *false positives* (have the same stem): policy → police

Porter Stemmer

- Algorithmic stemmer used in IR experiments since the 70s
- Consists of a series of rules designed to the longest possible suffix at each step
- Produces **stems not words**
- Makes a number of errors and difficult to modify
- Example of rules:
 - Replace *sses* by *ss* (e.g., stresses → stress).
 - Delete *s* if the preceding word part contains a vowel not immediately before the *s* (e.g., gaps → gap but gas → gas).
 - Replace *ied* or *ies* by *i* if preceded by more than one letter, otherwise by *ie* (e.g., ties → tie, cries → cri).
 - If suffix is *us* or *ss* do nothing (e.g., stress → stress).

```
>>> import os
>>> os.chdir('C:\\2026\\workshop1\\wk_solutions')
>>> os.getcwd()
'C:\\2026\\workshop1\\wk_solutions'
>>> from stemming.porter2 import stem
>>> stems=[stem(term) for term in terms]
>>> stems
['the', 'british', 'fashion', 'octob', 'royal', 'hall', 'will', 'albert', 'london', 'held', 'award']
```

- Porter2 stemmer addresses some of these issues, e.g., stem('policy') = 'polici', and stem('police') = 'polic' by using Porter2.

Krovetz Stemmer

- Hybrid algorithmic-dictionary
 - Word checked in dictionary
 - If present, either left alone or replaced with “exception”
 - If not present, word is checked for suffixes that could be removed
 - After removal, dictionary is checked again
- Produces words not stems
- Comparable effectiveness
- Lower false positive rate, somewhat higher false negative.
- It is difficult to determine which stemmer is “better” because effectiveness varies across datasets and tasks. The performance of IR models and machine learning algorithms can differ significantly depending on the characteristics of each data collection.

4. Feature Extraction: Phrases and N-grams

- **Phrases** are meaningful multi-word expressions that often represent a single concept. They can be identified through linguistic rules or statistical methods.
- Phrases are
 - More precise than single words
 - e.g., documents containing “black sea” vs. two words “black” and “sea”
 - Less ambiguous
 - e.g., “big apple” vs. “apple”
- But, phrase can be difficult for ranking documents when we use them [2]
 - e.g., Given query “fishing supplies”, how do we score documents with
 - exact phrase many times, exact phrase just once, individual words in same sentence, same paragraph, whole document, variations on words?
- Three possible approaches to recognize phrases:
 - Identify syntactic phrases using a *part-of-speech* (POS) tagger
 - Use word *n-grams*
 - Store word positions in indexes and use *proximity operators*

POS Tagging

- POS taggers use statistical models of text to predict syntactic tags of words, example tags:
 - NN (singular noun), NNS (plural noun), VB (verb), VBD (verb, past tense), VBN (verb, past participle), IN (preposition), JJ (adjective), CC (conjunction, e.g., “and”, “or”), PRP (pronoun), and MD (modal auxiliary, e.g., “can”, “will”).
- Phrases can then be defined as simple noun groups.

Original text:

Document will describe marketing strategies carried out by U.S. companies for their agricultural chemicals, report predictions for market share of such chemicals, or report market statistics for agrochemicals, pesticide, herbicide, fungicide, insecticide, fertilizer, predicted sales, market share, stimulate demand, price cut, volume of sales.

Brill tagger:

Document/NN will/MD describe/VB marketing/NN strategies/NNS carried/VBD out/IN by/IN U.S./NNP companies/NNS for/IN their/PRP agricultural/JJ chemicals/NNS ./, report/NN predictions/NNS for/IN market/NN share/NN of/IN such/JJ chemicals/NNS ./, or/CC report/NN market/NN statistics/NNS for/IN agrochemicals/NNS ./, pesticide/NN ./, herbicide/NN ./, fungicide/NN ./, insecticide/NN ./, fertilizer/NN ./, predicted/VBN sales/NNS ./, market/NN share/NN ./, stimulate/VB demand/NN ./, price/NN cut/NN ./, volume/NN of/IN sales/NNS ./.

- The POS tagger in the NLTK library outputs specific tags for words. The list of POS tags can be found at <https://medium.com/@muddaprince456/categorizing-pos-tagging-with-nltk-python-28f2bc9312c3>

```
> pip3 install nltk
```

```
>>> import nltk
```

```
>>> from nltk.tokenize import word_tokenize
```

You may need to do

```
>>> nltk.download('punkt')
```

```
>>> nltk.download('averaged_perceptron_tagger')
```

```
>>> text = word_tokenize("Document will describe marketing strategies carried out by U.S. companies for their agricultural chemicals, report predictions for market share of such chemicals, or report market statistics for agrochemicals, pesticide, herbicide, fungicide, insecticide, fertilizer, predicted sales, market share, stimulate demand, price cut, volume of sales.")
```

```
>>> text = [x for x in text if len(x)>1]
```

```
>>> text
```

```
['Document', 'will', 'describe', 'marketing', 'strategies', 'carried', 'out', 'by', 'U.S.', 'companies', 'for', 'their', 'agricultural', 'chemicals', 'report', 'predictions', 'for', 'market', 'share', 'of', 'such', 'chemicals', 'or', 'report', 'market', 'statistics', 'for', 'agrochemicals', 'pesticide', 'herbicide', 'fungicide', 'insecticide', 'fertilizer', 'predicted', 'sales', 'market', 'share', 'stimulate', 'demand', 'price', 'cut', 'volume', 'of', 'sales']
```

```
>>> pos_results = nltk.pos_tag(text)
```

```
>>> pos_results
```

```
[('Document', 'NNP'), ('will', 'MD'), ('describe', 'VB'), ('marketing', 'NN'), ('strategies', 'NNS'), ('carried', 'VBD'), ('out', 'RP'), ('by', 'IN'), ('U.S.', 'NNP'), ('companies', 'NNS'), ('for', 'IN'), ('their', 'PRP$'), ('agricultural', 'JJ'), ('chemicals', 'NNS'), ('report', 'VBP'), ('predictions', 'NNS'), ('for', 'IN'), ('market', 'NN'), ('share', 'NN'), ('of', 'IN'), ('such', 'JJ'), ('chemicals', 'NNS'), ('or', 'CC'), ('report', 'NN'), ('market', 'NN'), ('statistics', 'NNS'), ('for', 'IN'), ('agrochemicals', 'NNS'), ('pesticide', 'JJ'), ('herbicide', 'NN'), ('fungicide', 'JJ'), ('insecticide', 'JJ'), ('fertilizer', 'NN'), ('predicted', 'VBD'), ('sales', 'NNS'), ('market', 'NN'), ('share', 'NN'), ('stimulate', 'JJ'), ('demand', 'NN'), ('price', 'NN'), ('cut', 'NN'), ('volume', 'NN'), ('of', 'IN'), ('sales', 'NNS')]
```


N-Grams

- POS tagging too slow for large collections
- Simpler definition – phrase is any sequence of n words – known as *n-grams*
 - *bigram*: 2 word sequence, *trigram*: 3 word sequence, *unigram*: single words
 - N-grams also used at character level for applications such as OCR
- N-grams typically formed from *overlapping* sequences of words
 - i.e. move **n-word “window”** one word at a time in document

```
stems = ['the', 'british', 'fashion', 'octob', 'royal', 'hall', 'will', 'albert', 'london', 'held', 'award']
```

```
>>> bigrams = [stems[i]+' '+stems[i+1] for i in range(len(stems)-1)]
```

```
>>> bigrams
```

```
['the british', 'british fashion', 'fashion octob', 'octob royal', 'royal hall', 'hall will', 'will albert', 'albert london', 'london held', 'held award']
```

```
>>> trigrams = [stems[i]+' '+stems[i+1]+' '+stems[i+2] for i in range(len(stems)-2)]
```

```
>>> trigrams
```

```
['the british fashion', 'british fashion octob', 'fashion octob royal', 'octob royal hall', 'royal hall will', 'hall will albert', 'will albert london', 'albert london held', 'london held award']
```

CRICOS No.00213J

N-Grams cont.

- Frequent n-grams are more likely to be meaningful phrases
- N-grams form a Zipf distribution
 - Better fit than words alone
- Could index all n-grams up to specified length
 - Much faster than POS tagging
 - Uses a lot of storage
 - e.g., document containing 1,000 words would contain 3,990 instances of word n-grams of length $2 \leq n \leq 5$

5. Document Structure and Markup

- Some parts of documents are more important than others.
- Document parser recognizes structure using markup, such as HTML tags
 - Headers, anchor text, bolded text all likely to be important
 - Metadata can also be important
 - Links used for *link analysis*
- **Example of HTML files**

Tropical fish

Tropical fish include [fish](#) found in [tropical](#) environments around the world, including both [freshwater](#) and [salt water](#) species. [Fishkeepers](#) often use the term *tropical fish* to refer only those requiring fresh water, with saltwater tropical fish referred to as [marine fish](#).

Tropical fish are popular [aquarium](#) fish , due to their often bright coloration. In freshwater fish, this coloration typically derives from [iridescence](#), while salt water fish are generally [pigmented](#).

```
<html>
<head>
<meta name="keywords" content="Tropical fish, Airstone, Albinism, Algae eater,
Aquarium, Aquarium fish feeder, Aquarium furniture, Aquascaping, Bath treatment
(fishkeeping),Berlin Method, Biotope" />
...
<title>Tropical fish - Wikipedia, the free encyclopedia</title>
</head>
<body>
...
<h1 class="firstHeading">Tropical fish</h1>
...
<p><b>Tropical fish</b> include <a href="/wiki/Fish" title="Fish">fish</a> found in <a
href="/wiki/Tropics" title="Tropics">tropical</a> environments around the world,
including both <a href="/wiki/Fresh_water" title="Fresh water">freshwater</a> and <a
href="/wiki/Sea_water" title="Sea water">salt water</a> species. <a
href="/wiki/Fishkeeping" title="Fishkeeping">Fishkeepers</a> often use the term
<i>tropical fish</i> to refer only those requiring fresh water, with saltwater tropical fish
referred to as <i><a href="/wiki/List_of_marine_aquarium_fish_species" title="List of
marine aquarium fish species">marine fish</a></i>.</p>
<p>Tropical fish are popular <a href="/wiki/Aquarium" title="Aquarium">aquarium</a>
fish , due to their often bright coloration. In freshwater fish, this coloration typically
derives from <a href="/wiki/Iridescence" title="Iridescence">iridescence</a>, while salt
water fish are generally <a href="/wiki/Pigment" title="Pigment">pigmented</a>.</p>
...
</body></html>
```

Html tags

- HTML files show more structure, where each field or element in HTML is indicated by a start tag (such as **<h1>**) and an optional end tag (e.g., **</h1>**).
- Elements can have attributes (with values), given by attribute_name = "value" pairs.
- The **<head>** element contains metadata that is not displayed by a browser.
- The main heading is indicated by the **<h1>** tag, and terms that should be displayed in bold or italic are indicated by **** or **<i>** tags etc.
- Hyperlinks are commonly used, such as ** fish **, where "fish" is the anchor text.
- An anchor text is used as a description of the content of the *destination page*. Anchor text tends to be short, descriptive, and similar to query text.
- Retrieval experiments have shown that anchor text has significant impact on effectiveness for *some types of queries*.

CRICOS No.001131

```
from html.parser import HTMLParser
class Parser(HTMLParser):
    # method to append the start tag to the list start_tags.
    def handle_starttag(self, tag, attrs):
        global start_tags
        global attrs_names
        if tag=='a':
            start_tags.append(tag)
            attrs_names.append(attrs)
start_tags = []
attrs_names = []
# Creating an instance of our class.
parser = Parser()
# open a html file
myHtml=open('html_example.html')
file_html=myHtml.read()

# Providing the input.
parser.feed(file_html)
#print("start tags:", start_tags)
#print("attributes:", attrs_names)
hyper_links = [x[0][1] for x in attrs_names]
print(hyper_links)
myHtml.close()
```

6. Review Questions and Workshops

- This unit provides weekly review questions from Week 2 through Week 11.
- These serve as the main reference resources to help you prepare for the final exam.
- You should complete them on your own weekly, then compare your answers with the solutions and contact the teaching team if you have any questions.
- Week 2 provides 4 review questions:
- **Question 1 (Text statistic)**
- **Question 2** Which of the following is FALSE? and explain why it is FALSE.
- **Question 3 (N-grams)**
- **Question 4 (Html tags)**

CRICOS N6.00213J

```
myfile=open('6146.xml')
file_=myfile.readlines()
```

```
doc={}
word_count=0
```

```
text = [line.replace("<p>", "").replace("</p>", "").replace("\n", "") for line in file_ if line.startswith('<p>')]
print(text)
```

```
['Argentine bonds were slightly higher in a small technical bounce Wednesday amid low volume.', 'A trader at a large foreign bank said there was a slight technical bounce at the opening, and he did not expect prices to change much during the session as no market-moving news is expected.', 'The 5.5 percent dollar-denominated Bocon Previsional 2 due 2001 rose $0.15 to 115.15. Argentina's FRB due 2005 rose 1/8 to 77-3/8.', '"There is general uncertainty," said the trader, pointing to all the events the market is waiting for, including the passage of the government's new economic measures through Congress, which is now not expected until early October.', 'In addition, traders are awaiting a meeting Friday between Economy Minister Roque Fernandez and an International Monetary Fund delegation on Argentina's fiscal deficit.', '-- Axel Bugge, Buenos Aires Newsroom, 541 318-0668']
```

```
for line in text:
    for term in line.split():
        word_count += 1
        term = term.lower()
        if len(term) > 2:
            try:
                doc[term] += 1
            except KeyError:
                doc[term] = 1
print(word_count)
```

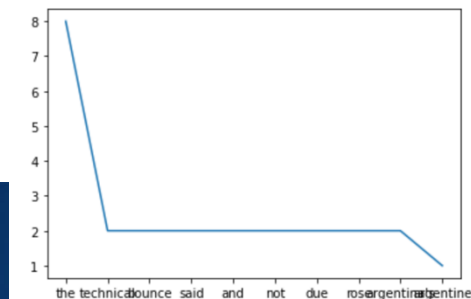
137

Sort the dictionary and plot the frequency distribution

```
import matplotlib.pyplot as plt
myList = doc.items()
myList = sorted(myList, key=lambda x:x[1], reverse=True) #sorted by value
myLList = list(myList)
myList = myLList[:10]
print(myList)
x, y = zip(*myList)
```

```
plt.plot(x, y)
plt.show()
```

```
[('the', 8), ('technical', 2), ('bounce', 2), ('said', 2), ('and', 2), ('not', 2), ('due', 2), ('rose', 2), ('argentina', 2), ('argentine', 1)]
```



Workshops and Practical Tasks

- Workshops commence this week. Please attend the room for which you registered and meet your tutor, as well as potential teammates for Assignment 2. This assignment must be completed in groups of three students within the same workshop.
- The workshops are designed to ensure you are familiar with methods for text processing and some applications.
- Solutions will be provided in both PDF and Python code formats. To support your study effectively, please note the following points for **How to Use Solutions**:
 - PDF solutions can be used to compare your answers or provide hints when you encounter difficulties.
 - Code solutions should only be used if you are completely stuck after putting in your own effort. They are intended to help you understand the concepts and provide a quick way to implement them in Python.
 - We encourage you to design and implement all answers yourself and discuss any issues with your tutor, team members, or classmates.
 - If no specific requirements are provided for a task or question, you may use the knowledge you acquired in this unit to develop your own solutions or consult the unit coordinator if your answer differs significantly from the provided solutions.
- **Important Reminder**
 - It is very important to complete all tasks and questions weekly. Catching up at the end of the semester is extremely difficult.

References

- [1] Z. Hong *et al.*, "Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL," in *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 12, pp. 7328-7345, Dec. 2025, doi: 10.1109/TKDE.2025.3609486.
- [2] Chapter 4 in book - W. Bruce Croft, *Search Engines - Information retrieval in Practice*; Pearson, 2010.
- [3] M. Albathan, **Y. Li**, Y. Xu, Using extended random set to find specific patterns, in *Proceedings of 2014 IEEE/WIC/ACM International Conference on Web Intelligence* (Vol. 2), 11–14 August 2014, Warsaw, Poland, 2014, pp. 30-37 (**Best Student Paper Award**, <https://dl.acm.org/citation.cfm?id=2682826>)